

Non-Gaussian Response Distributions

GAM.jl Contributors

Table of contents

Overview	1
Setup	2
Poisson GAM — count data	2
Simulating count data	2
Fitting the model	3
Smooth estimate	4
Deviance residuals	5
Binomial GAM — binary data	6
Simulating binary data	6
Fitting the model	7
Smooth estimate	8
Deviance residuals	9
Gamma GAM — positive continuous data	10
Simulating positive continuous data	10
Fitting the model	11
Smooth estimate	12
Deviance residuals	13
Comparison	14
All smooth estimates together	15
Summary	15

Overview

GAMs are not limited to Gaussian responses. By specifying a **family** (distribution) and **link function**, we can model count data, binary outcomes, positive continuous data, and more. The general model is:

$$g(\mu_i) = \beta_0 + f_1(x_{1i}) + \cdots + f_p(x_{pi}), \quad y_i \sim \text{Family}(\mu_i, \phi)$$

This vignette demonstrates three common non-Gaussian families:

Family	Link	Response type	Example
Poisson	log	Counts	Species counts, event rates
Binomial	logit	Binary / proportions	Presence-absence, disease status
Gamma	inverse (or log)	Positive continuous	Waiting times, claim sizes

Setup

```
using GAM
using StatsAPI: residuals, fitted, r2, deviance
using Statistics: mean
using GLM: LogLink, LogitLink

using DataFrames
using Random
using Plots
using Distributions
```

Poisson GAM — count data

Simulating count data

We simulate counts from a Poisson distribution with a smooth log-rate:

$$y_i \sim \text{Poisson}(\mu_i), \quad \log(\mu_i) = 1 + 1.5 \sin(2\pi x_i)$$

```
Random.seed!(42)
n = 300
x = sort(rand(n))
    = 1.0 .+ 1.5 .* sin.(2 .* x)
    = exp.( )
y_pois = rand.(Poisson.( ))

df_pois = DataFrame(x = x, y = Float64.(y_pois))
```

	x	y
	Float64	Float64
1	0.000715255	1.0
2	0.00119567	3.0
3	0.0100473	4.0
4	0.016095	3.0
5	0.0269848	4.0
6	0.0301856	4.0
7	0.0369673	5.0
8	0.0379955	2.0
9	0.040211	5.0
10	0.0424826	5.0
11	0.0467336	3.0
12	0.0467959	4.0
13	0.0484837	3.0
14	0.0513185	4.0
15	0.0552982	5.0
16	0.055968	4.0
17	0.0588083	6.0
18	0.0589488	5.0
19	0.0604437	1.0
20	0.0651857	2.0
21	0.066672	9.0
22	0.0726606	10.0
23	0.0786647	7.0
24	0.0793339	4.0
...	...	...

Fitting the model

```
m_pois = gam(@gam_formula(y ~ s(x, k = 15, bs = :cr)), df_pois;
             family = Poisson(), link = LogLink())
m_pois
```

Generalized Additive Model

Formula: y ~ 1

Family: Poisson

Link: LogLink

Method: REML

Parametric coefficients:

	Coef.	Std. Error	z	Pr(> z)
(Intercept)	1.11917	0.040866	27.39	<1e-99

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x,bs=cr)	7.45	14

R^2 (adj) = 0.759 Deviance explained = 76.5%
n = 300

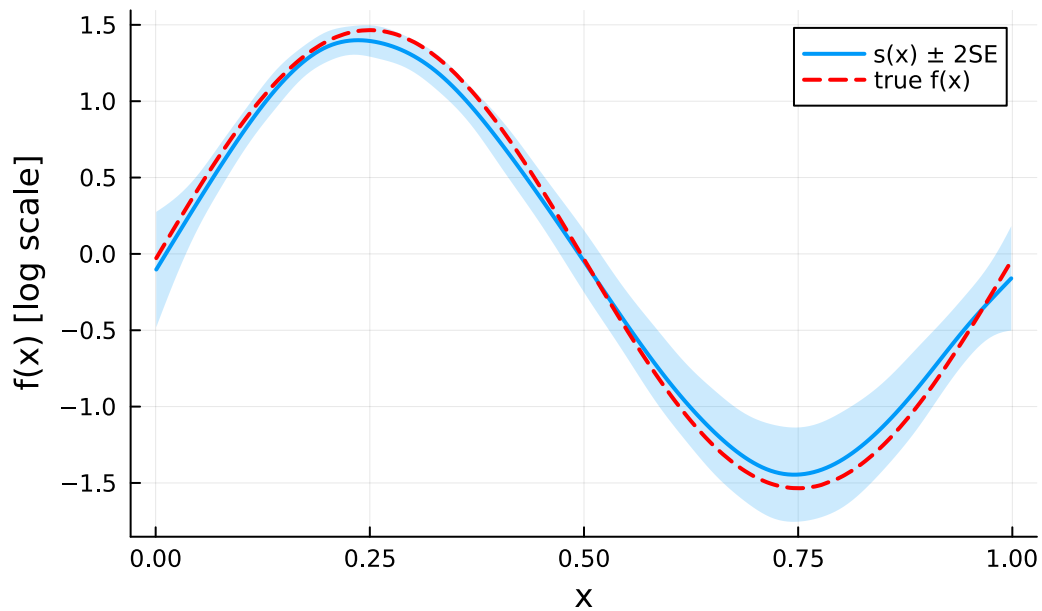
Smooth estimate

```
se_pois = smooth_estimates(m_pois; n = 200)
x_grid = se_pois.covariates[:x]

p1 = plot(x_grid, se_pois.estimate;
          ribbon = 2 .* se_pois.se,
          fillalpha = 0.2,
          label = "s(x) ± 2SE",
          linewidth = 2,
          xlabel = "x",
          ylabel = "f(x) [log scale]",
          title = "Poisson GAM - smooth on link scale")
plot!(p1, x,  .- mean();
      label = "true f(x)",
      linestyle = :dash,
      linewidth = 2,
      color = :red)

p1
```

Poisson GAM — smooth on link scale

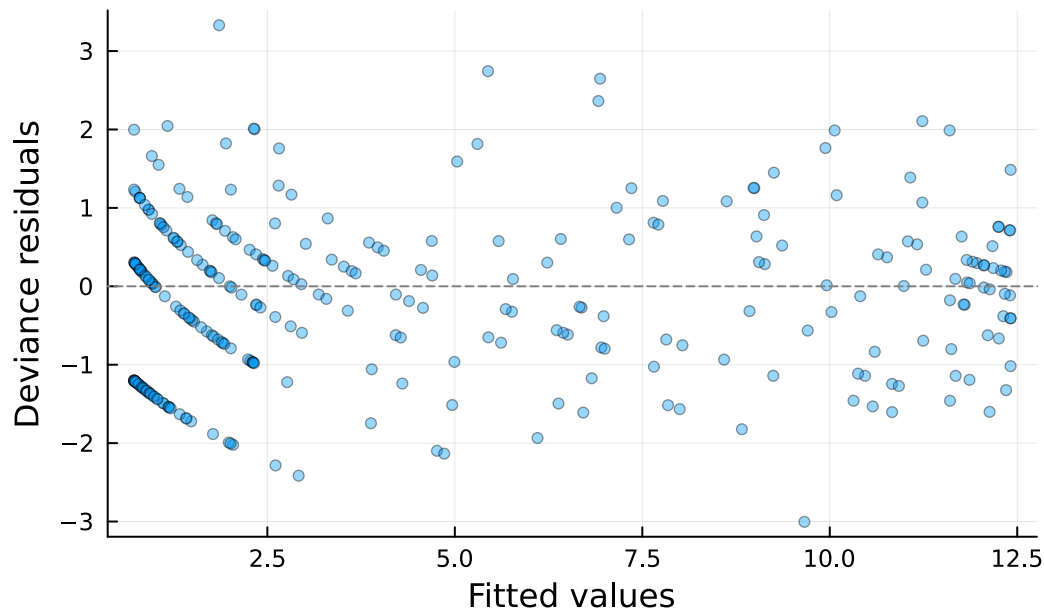


Deviance residuals

```
resid_pois = residuals(m_pois; type = :deviance)

p2 = scatter(fitted(m_pois), resid_pois;
             xlabel = "Fitted values",
             ylabel = "Deviance residuals",
             title = "Poisson GAM - residuals",
             alpha = 0.4,
             markersize = 3,
             label = false)
hline!(p2, [0]; linestyle = :dash, color = :gray, label = false)
p2
```

Poisson GAM — residuals



Binomial GAM — binary data

Simulating binary data

We simulate binary outcomes from a logistic model:

$$y_i \sim \text{Bernoulli}(p_i), \quad \text{logit}(p_i) = -0.5 + 2 \sin(2\pi x_i)$$

```
Random.seed!(42)
x_bin = sort(rand(n))
_bin = -0.5 .+ 2.0 .* sin.(2 .* x_bin)
p_true = 1.0 ./ (1.0 .+ exp.(-_bin))
y_bin = Float64.(rand.(Bernoulli.(p_true)))

df_bin = DataFrame(x = x_bin, y = y_bin)
```

	x	y
	Float64	Float64
1	0.000715255	0.0
2	0.00119567	0.0
3	0.0100473	0.0
4	0.016095	1.0
5	0.0269848	0.0
6	0.0301856	1.0
7	0.0369673	0.0
8	0.0379955	1.0
9	0.040211	1.0
10	0.0424826	0.0
11	0.0467336	1.0
12	0.0467959	0.0
13	0.0484837	0.0
14	0.0513185	0.0
15	0.0552982	1.0
16	0.055968	1.0
17	0.0588083	1.0
18	0.0589488	0.0
19	0.0604437	1.0
20	0.0651857	1.0
21	0.066672	1.0
22	0.0726606	0.0
23	0.0786647	0.0
24	0.0793339	1.0
...	...	...

Fitting the model

```
m_bin = gam(@gam_formula(y ~ s(x, k = 15, bs = :cr)), df_bin;
            family = Binomial(), link = LogitLink())
m_bin
```

Generalized Additive Model

Formula: y ~ 1

Family: Binomial

Link: LogitLink

Method: REML

Parametric coefficients:

	Coef.	Std. Error	z	Pr(> z)
(Intercept)	-0.475796	0.147233	-3.23	0.0012

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x,bs=cr)	5.01	14

R^2 (adj) = 0.244 Deviance explained = 25.7%
n = 300

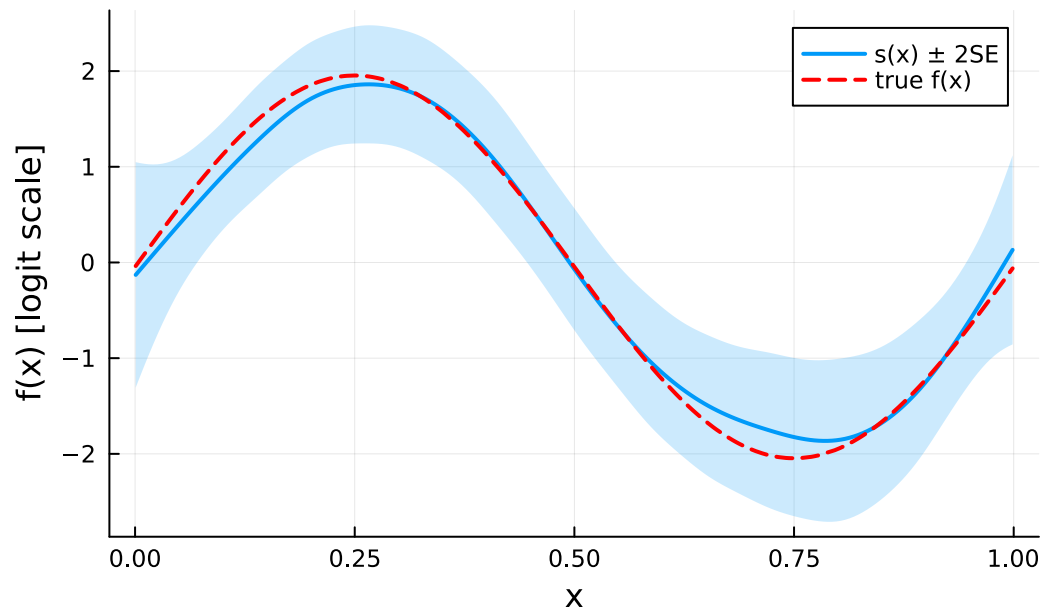
Smooth estimate

```
se_bin = smooth_estimates(m_bin; n = 200)
x_grid_bin = se_bin.covariates[:x]

p3 = plot(x_grid_bin, se_bin.estimate;
  ribbon = 2 .* se_bin.se,
  fillalpha = 0.2,
  label = "s(x) ± 2SE",
  linewidth = 2,
  xlabel = "x",
  ylabel = "f(x) [logit scale]",
  title = "Binomial GAM - smooth on link scale")
plot!(p3, x_bin, _bin .- mean(_bin);
  label = "true f(x)",
  linestyle = :dash,
  linewidth = 2,
  color = :red)

p3
```

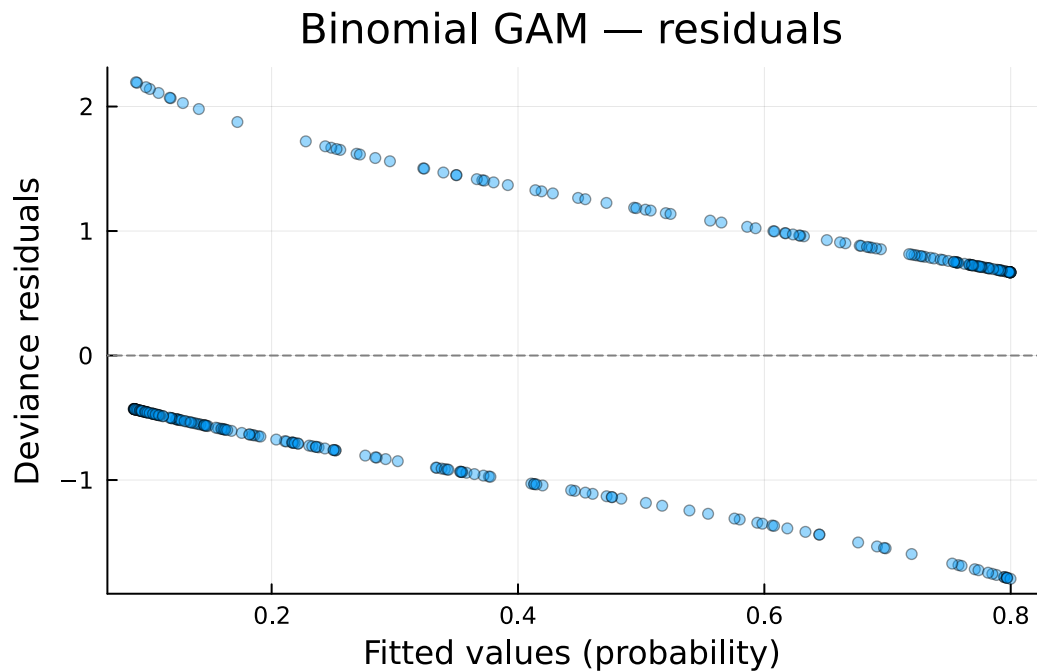

Binomial GAM — smooth on link scale



Deviance residuals

```
resid_bin = residuals(m_bin; type = :deviance)

p4 = scatter(fitted(m_bin), resid_bin;
  xlabel = "Fitted values (probability)",
  ylabel = "Deviance residuals",
  title = "Binomial GAM - residuals",
  alpha = 0.4,
  markersize = 3,
  label = false)
hline!(p4, [0]; linestyle = :dash, color = :gray, label = false)
p4
```



Gamma GAM — positive continuous data

Simulating positive continuous data

We simulate positive continuous data from a Gamma distribution with a smooth log-mean:

$$y_i \sim \text{Gamma}(\text{shape}, \text{scale}_i), \quad \log(\mu_i) = 1 + \sin(2\pi x_i)$$

```
Random.seed!(42)
x_gam = sort(rand(n))
_gam = 1.0 .+ sin.(2 .* x_gam)
_gam = exp.(_gam)
shape = 5.0
y_gam = rand.(Gamma.(shape, _gam ./ shape))

df_gam = DataFrame(x = x_gam, y = y_gam)
```

	x	y
	Float64	Float64
1	0.000715255	1.57074
2	0.00119567	4.44249
3	0.0100473	4.49489
4	0.016095	5.16294
5	0.0269848	2.41209
6	0.0301856	2.23542
7	0.0369673	1.97537
8	0.0379955	3.81063
9	0.040211	1.79205
10	0.0424826	4.17583
11	0.0467336	3.6381
12	0.0467959	2.23194
13	0.0484837	2.99005
14	0.0513185	5.47756
15	0.0552982	1.76053
16	0.055968	2.78088
17	0.0588083	3.21451
18	0.0589488	2.85169
19	0.0604437	7.78466
20	0.0651857	3.49634
21	0.066672	3.49746
22	0.0726606	1.83008
23	0.0786647	1.81257
24	0.0793339	8.44885
...	...	...

Fitting the model

We use `Gamma()` with `LogLink()` (log link is more commonly used in practice than the canonical inverse link):

```
m_gam = gam(@gam_formula(y ~ s(x, k = 15, bs = :cr)), df_gam;
             family = Gamma(), link = LogLink())
m_gam
```

Generalized Additive Model

Formula: $y \sim 1$

Family: Gamma
Link: LogLink
Method: REML

Parametric coefficients:

	Coef.	Std. Error	z	Pr(> z)
(Intercept)	1.04246	0.057735	18.06	<1e-72

Approximate significance of smooth terms:

Smooth	edf	Ref.df
s(x,bs=cr)	5.20	14

R^2 (adj) = 0.716 Deviance explained = 72.1%
n = 300

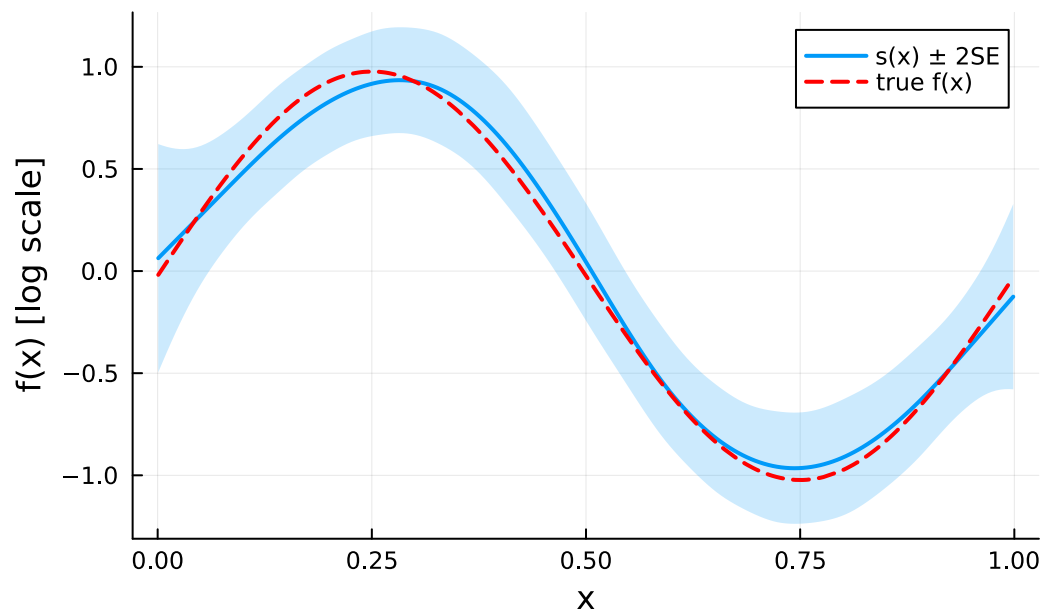
Smooth estimate

```
se_gam = smooth_estimates(m_gam; n = 200)
x_grid_gam = se_gam.covariates[:x]

p5 = plot(x_grid_gam, se_gam.estimate;
          ribbon = 2 .* se_gam.se,
          fillalpha = 0.2,
          label = "s(x) ± 2SE",
          linewidth = 2,
          xlabel = "x",
          ylabel = "f(x) [log scale]",
          title = "Gamma GAM - smooth on link scale")
plot!(p5, x_gam, _gam .- mean(_gam);
      label = "true f(x)",
      linestyle = :dash,
      linewidth = 2,
      color = :red)

p5
```

Gamma GAM — smooth on link scale

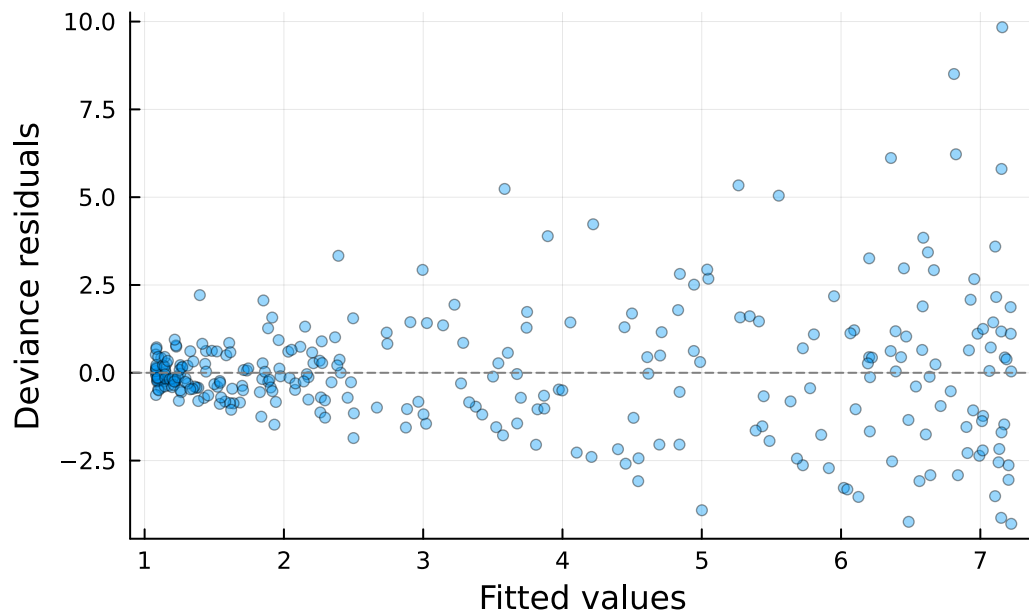


Deviance residuals

```
resid_gam = residuals(m_gam; type = :deviance)

p6 = scatter(fitted(m_gam), resid_gam;
  xlabel = "Fitted values",
  ylabel = "Deviance residuals",
  title = "Gamma GAM - residuals",
  alpha = 0.4,
  markersize = 3,
  label = false)
hline!(p6, [0]; linestyle = :dash, color = :gray, label = false)
p6
```

Gamma GAM — residuals



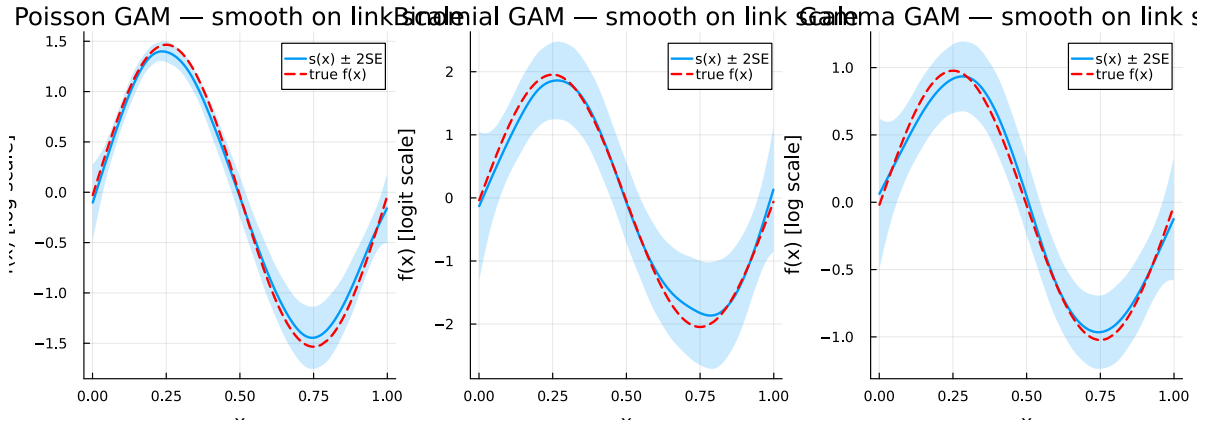
Comparison

```
println("Family      EDF      Dev.Expl(%)  Scale")
println(" " ^ 55)
for (name, m) in [("Poisson", m_pois), ("Binomial", m_bin), ("Gamma", m_gam)]
  e = round(edf(m)[1]; digits = 2)
  de = round(r2(m) * 100; digits = 1)
  sc = round(m.scale; digits = 4)
  println("$ (rpad(name, 14)) $(lpad(string(e), 6))  $(lpad(string(de), 12))  $(lpad(string(sc), 12))")
end
```

Family	EDF	Dev.Expl(%)	Scale
Poisson	7.45	76.5	1.0
Binomial	5.01	25.7	1.0
Gamma	5.2	72.1	1.0

All smooth estimates together

```
plot(p1, p3, p5; layout = (1, 3), size = (1000, 350))
```



Summary

In this vignette we:

1. Simulated count data and fitted a **Poisson GAM** with a log link
2. Simulated binary data and fitted a **Binomial GAM** with a logit link
3. Simulated positive continuous data and fitted a **Gamma GAM** with a log link
4. Examined smooth estimates and deviance residuals for each family
5. Compared EDF and deviance explained across families

Each family uses a different link function to map the linear predictor to the mean of the response distribution, but the smooth estimation machinery is the same.